Transaction Management

Transaction Types

Transactions are operations in a Mule app for which the result cannot remain indeterminate. When a series of steps in a flow must succeed or fail as one unit, Mule uses a transaction to demarcate that unit.

For example, you might use a transaction to encapsulate several steps in a flow that result in committing information to a database. In this type of scenario, the commit (or transaction) is either entirely complete and succeeds, or is incomplete and fails. Even if partially complete, the commit fails. When a transaction fails, Mule rolls back the operations within the transaction so that no part results in partial completion.

Implement a Reliability Pattern to design your application so it is capable of reliable messaging, even if the application receives messages from a non-transactional connector.

You can also configure Bitronix to manage transactions in your Mule application.

Transaction Types

Mule supports Single Resource (Local, the default) and Extended Architecture (XA) transaction types (`transactionType`). The only components that can define the

transaction type are event sources (for example, `jms:listener` and `vm:listener`) and the Try scope.

The following table describes the characteristics of each transaction type and the requisites for an operation to join the transaction:

Transaction Type	Characteristics	Requisites to Join the Transaction
Single Resource (Local)	• Performs better than XA transactions. • Does not support nested transactions.	• The operation must support transactions. • All operations inside the transaction must belong to the same Connector. (For example `jms:listener`, `jms:consume` and `jms:publish`). • All operations must use the same Global Configuration (`config-ref`).
Extended Architecture (XA)	• Involves using a two-phase commit protocol (2PC). • Slower but more reliable than Single Resource transactions. • Supports nested transactions.	• The operation must support transactions.

Transactional Actions

A Transactional Action (`transactionalAction`) defines the type of action that operations take regarding transactions. The following table describes all available transactional actions:

Action	Behavior	Available in
`ALWAYS_BEGIN`	Always start a new transaction when receiving a message. If a Single Resource transaction exists, an error occurs. If an XA transaction exists, a nested transaction is created.	• Listeners that support transactions • Try scope
`ALWAYS_JOIN`	Always expect a transaction to be in progress when a message is received. If there is no transaction, an error occurs.	• Operations that support transactions
`BEGIN_OR_JOIN`	If a transaction is already in progress when a message is received, join the transaction if possible. Otherwise, start a new transaction.	• Try scope
`JOIN_IF_POSSIBLE`	Join the current transaction if one is available. Otherwise, no transaction is created.	• Operations that support transactions
`INDIFFERENT`	Do not treat actions as a transaction.	• Try scope
`NONE`	Do not initiate a transaction.	• Listeners that support transactions

Action	Behavior	Available in
`NOT_SUPPORTED`	Execute outside any existent transaction.	• Operations that support transactions

Transaction Timeout

The transaction timeout defines the maximum time, in milliseconds, that a transaction can remain active before Mule rolls it back. The default value is `30000` (30 seconds).

To change the default transaction timeout for all transactions in your Mule app, set the `defaultTransactionTimeout` attribute in the global `configuration` element:

```xml
<configuration defaultTransactionTimeout="60000"/>
```

For more information, see Global Settings.

If you use Bitronix to manage XA transactions, Bitronix also uses its own `mule.bitronix.transactiontimeout` property, which is set in seconds and defaults to `60`. See Using Bitronix to Manage Transactions.

Configuring a Transaction in the Event

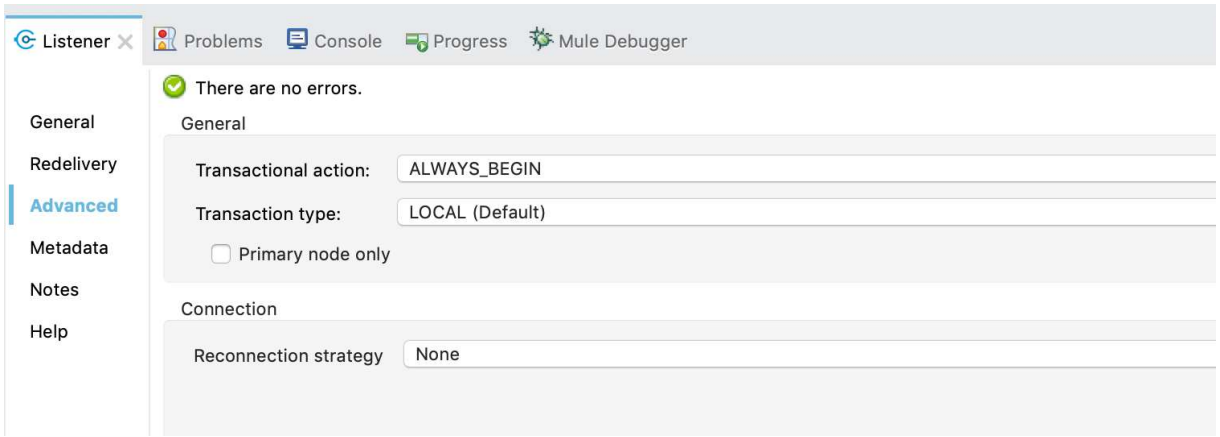
Source

You can start a transaction from a Mule event source. In this case, the entire flow becomes a transaction. This is useful when working with messaging connectors to prevent the consumption of the message if a problem occurs when processing it, allowing you to retry later (because of the rollback).

To initiate a transaction from a Mule event source, configure its Transaction type and Transactional action:

- In Anypoint Studio

  Open the Listener's **Advanced** tab, and set the **Transaction type** and the **Transactional action** values:



- In the Configuration XML

  Add the `transactionalAction` element and the `transactionType` element (if necessary), and set their values.

  The XML example below illustrates a `vm:listener` event source configured to initiate a Single Resource (Local) transaction:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<mule xmlns:http="http://www.mulesoft.org/schema/mule/http" xmlns:vm="http://www.mu
        xmlns="http://www.mulesoft.org/schema/mule/core"
        xmlns:doc="http://www.mulesoft.org/schema/mule/documentation" xmlns:xsi="ht
http://www.mulesoft.org/schema/mule/vm http://www.mulesoft.org/schema/mule/vm/curre
http://www.mulesoft.org/schema/mule/http http://www.mulesoft.org/schema/mule/http/c

        <vm:config name="VM_Config1" >
                <vm:queues >
                        <vm:queue queueName="input" />
                        <vm:queue queueName="output" />
                </vm:queues>
        </vm:config>

        <flow name="source-transactionsFlow">
                <vm:listener config-ref="VM_Config1" queueName="input" transactiona
                <http:request method="GET" url="www.google.com"/>
```

Expand content

Configuring a Transaction in a Try Scope

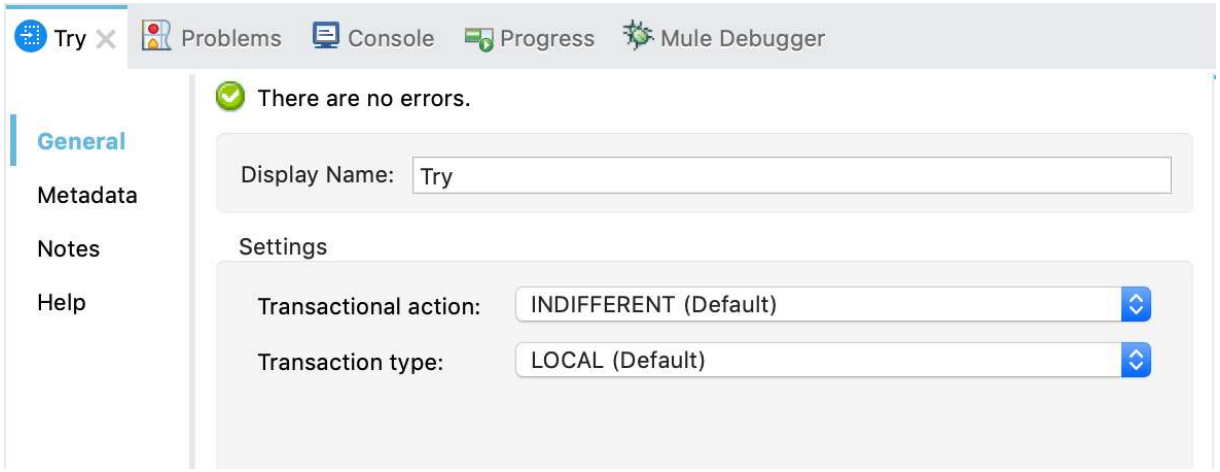
A Mule flow can also begin with a non-transactional connector (such as HTTP) that requires a transaction within the flow. For example, a Mule flow might accept information from an external Web service, then transform the data before charging

a credit card and saving invoice information to a database. In such a situation, you use the Try scope to set up a transaction by wrapping the credit card charge and database commit operations within a transaction to ensure either complete success or complete failure and rollback.

You can configure a transaction in a Try scope component by setting a Transaction type and Transactional action:

- In Anypoint Studio

  Open the Try scope's **General** tab, and set the **Transaction type** and the **Transactional action** values:



- In the Configuration XML

  Add the `transactionalAction` element and the `transactionType` element (if necessary), and set their values.

  In the following XML example the Try scope demarcates a transaction that includes a Database operation (`db:insert`) and a VM operation (`vm:publish`). Either both will be applied if the entire process succeeds, or none will be applied if an error occurs within the Try scope:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<mule xmlns:vm="http://www.mulesoft.org/schema/mule/vm" xmlns:db="http://www.mulesc
        xmlns="http://www.mulesoft.org/schema/mule/core"
        xmlns:doc="http://www.mulesoft.org/schema/mule/documentation"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="h
http://www.mulesoft.org/schema/mule/db http://www.mulesoft.org/schema/mule/db/curre
http://www.mulesoft.org/schema/mule/vm http://www.mulesoft.org/schema/mule/vm/curre
        <db:config name="Database_Config">
                <db:derby-connection database="myDb" create="true" />
        </db:config>
        <vm:config name="VM_Config">
                <vm:queues>
                        <vm:queue queueName="myQueue" />
                </vm:queues>
        </vm:config>
        <flow name="transactionsFlow">
                <try transactionalAction="ALWAYS_BEGIN" transactionType="XA">
```

Expand content

How Transactions Affect Scopes and Routers

When running in a transactional scope, the entire process must run in the same thread. This can change the way a scope executes, or how a transaction is handled.

Below is detailed the behavior of the scopes and routers you must consider when running in a transactional scope:

Scopes

- Async: When running within a transaction, the Async scope will still run in another thread (remaining asynchronous). However, this means that the entire execution of the processors within this scope is out of the transactional scope. Therefore, any error produced inside an Async scope does not result in a rollback.
- Until Successful: When running within a transaction, the Until Successful scope blocks the thread. This means that the thread used by this execution cannot be used to process any other request while the Until Successful scope is performing the delay between retries.
- Parallel Foreach: When running within a transaction, the Parallel Foreach scope does not execute in parallel. This means it executes as the Foreach scope: the second element of the collection is processed after the first one has finished. This does not affect the way this scope handles errors.
- Batch Processing: Since Batch Processing is designed to work on parallel as the Async scope, and every record is treated transactionally (with internal Batch transactions), the batch execution is not part of the transaction.

Routers

- Scatter Gather: When running within a transaction, Scatter Gather does not execute in parallel. This means that the second route is executed after the first one is processed, the third after the second one, etc. This does not affect the way this component handles errors.
- Flow Reference: If the flow (or subflow) being executed is running within a transaction, the execution of the flow (or subflow) referenced by the Flow Reference component continues

with the same transaction. If the referenced flow has a Mule event source that begins a transaction (For example a `jms:listener` source with `transactionalAction="ALWAYS_BEGIN"`), the `transactionalAction` is ignored. The Flow Reference component executes the referenced flow components except for its event source.

Any other router or scope (for example, Foreach) continues with the transaction and its behavior is not modified.

Error Handling

When an error occurs during a transaction, your application must either handle the error and continue or perform a rollback.

There are two types of error handlers, which behave differently when an error occurs during a transaction:

- On Error Propagate
  - If the `on-error-propagate` error handler is inside the `error-handler` scope corresponding to the component that began the transaction:

    The transaction is rolled back before executing the processors of the `on-error-propagate` scope. This means that the processors inside the error handler do not run within the transaction.
  - If the `on-error-propagate` error handler is inside an element that did not start the transaction:

    The transaction is not rolled back and the processors inside the `on-error-propagate` error handler run within the transaction. Remember that some scopes and routers behave differently when executing within a transaction.
- On Error Continue

The error is handled, the transaction remains active and is able to commit. The processors inside the `on-error-continue` run within the transaction.

Example: Error Occurs in Try Scope

Consider an example in which the event source (`jms:listener`) starts a transaction, which is continued by the Try scope (`transactionalAction` is set to default, `INDIFFERENT`).

When the error occurs, it is handled by the `error-handler` of the Try scope first. The error is handled by an `on-error-propagate` error handler, but the transaction is not rolled back because it was not the Try scope that started the transaction. Only the flow error handler can roll back the transaction, because the transaction was initiated at flow level (from its event source).

The `jms:publish` operation inside the `on-error-propagate` error handler joins the transaction. The error is then propagated to the flow `error-handler`, which handles the error with an `on-error-continue` error handler. The `jms:consume` operation inside the `on-error-continue` error handler runs within the transaction. Finally, the transaction is committed.

XML |

```xml
<flow name="someFlowWithTx">
        <jms:listener config-ref="JMS_Config" destination="test.in" transactionalActi
        <!-- Processors -->
        <try>
                <raise-error type="APP:SOME"/>
                <error-handler>
                        <on-error-propagate>
                                <jms:publish config-ref="JMS_Config" destination="tes
                        </on-error-propagate>
                </error-handler>
```

```
        </try>
        <error-handler>
                <on-error-continue>
                        <jms:consume config-ref="JMS_Config" destination="test.in2" t
                </on-error-continue>
        </error-handler>
</flow>
```

If the `on-error-continue` error handler is replaced with an `on-error-propagate` error handler in the previous example, then the transaction is rolled back before the execution of the processors. In this scenario, the configuration is not correct because the `jms:consume` operation cannot join any transaction.

Example: Error Occurs in a Referenced Flow

The following example behaves the same as the previous one, but instead of having a Try scope it uses a Flow Reference and the error occurs in the referenced flow:

XML

```
<flow name="someFlowWithTx">
        <jms:listener config-ref="JMS_Config" destination="test.in" transactionalActi
        <!-- Processors -->
        <flow-ref name="someFlowContinuesTx"/>
        <error-handler>
                <on-error-continue>
                        <jms:consume config-ref="JMS_Config" destination="test.in2" t
                </on-error-continue>
        </error-handler>
</flow>
```

```xml
<flow name="someFlowContinuesTx">
        <raise-error type="APP:SOME"/>
        <error-handler>
                <on-error-propagate>
                        <jms:publish config-ref="JMS_Config" destination="test.out" t
                </on-error-propagate>
        </error-handler>
</flow>
```

Because the `someFlowContinuesTx` flow did not initiate the transaction, its error handler does not roll back the transaction.

Example: Comparing Error Handlers

This example shows the differences between the `on-error-continue` error handler and the `on-error-propagate` error handler:

XML

```xml
<flow name="flow1">
        <jms:listener config-ref="JMS_Config" destination="test.in" transactionalActi
        <raise-error type="APP:SOME"/>
        <error-handler>
                <on-error-continue>
                        <try transactionalAction="ALWAYS_BEGIN">
                                <logger message="hello"/>
                        </try>
                </on-error-continue>
        </error-handler>
</flow>
```

```
<flow name="flow2">
        <jms:listener config-ref="JMS_Config" destination="test.in" transactionalActi
        <raise-error type="APP:SOME"/>
        <error-handler>
```

Expand content

In `flow1`, the `on-error-continue` error handler keeps the transaction active, but the Try scope inside it tries to initiate a new transaction. The configuration is not valid and results in an error.

In `flow2`, the `on-error-propagate` error handler rolls back the transaction, the processors inside the `on-error-propagate` error handler are executed, and a new transaction is created. This configuration is correct.

See Also

- Try Scope
- Reliability Patterns
- On-Error Scope

Back to top

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

Yes No

Legal | Terms of Service | Privacy | Trust | Contact | Responsible Disclosure | Cookie Settings
Your Privacy Choices

© Copyright 2026 Salesforce, Inc. All rights reserved. Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States